

Week 6: Designing Scalable Microservices!?

What does that even LOOK LIKE!?!?



What have we looked at so far?!

- your own mental model of "distributed systems"?!
 - Conceptual representations... processes, threads, containers, virtual machines, infrastructure as code, stateful/stateless services, load testing, service APIs, load balancing (and more!)
 - Concrete mechanisms... docker, terraform, Locust, ECR/ECS, OpenAPI, .yaml, ... (and more)!
- Draw some pictures!
 - What maps to what?!
 - Why are containers so important?
 - How do they relate to microservices!?

Week 5 in review

- Topics we (briefly!) covered!
 - Service API Design
 - CRUD Pattern
 - OpenAPI/Swagger
 - Application Service Containers
 - Horizontal Scaling
 - Load Balancing

Grafana

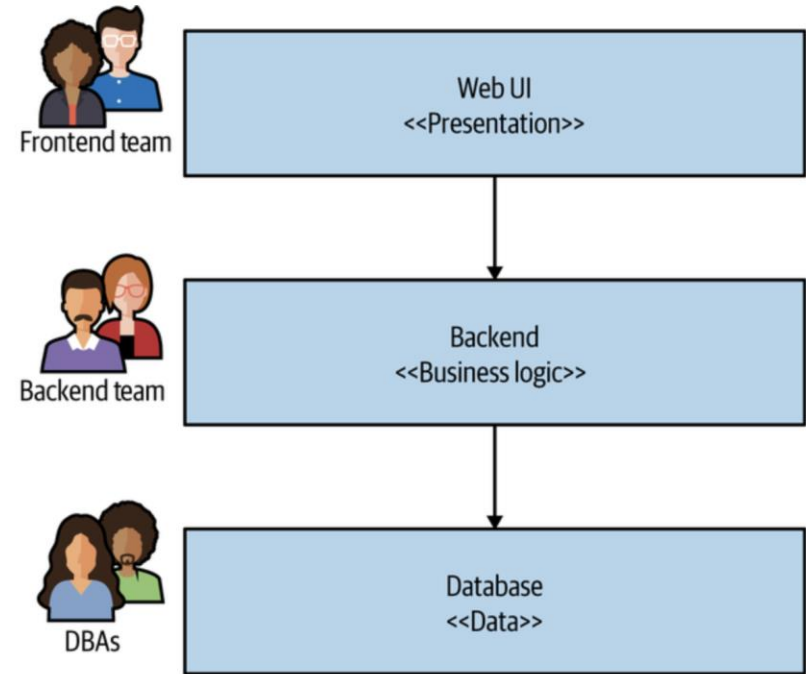
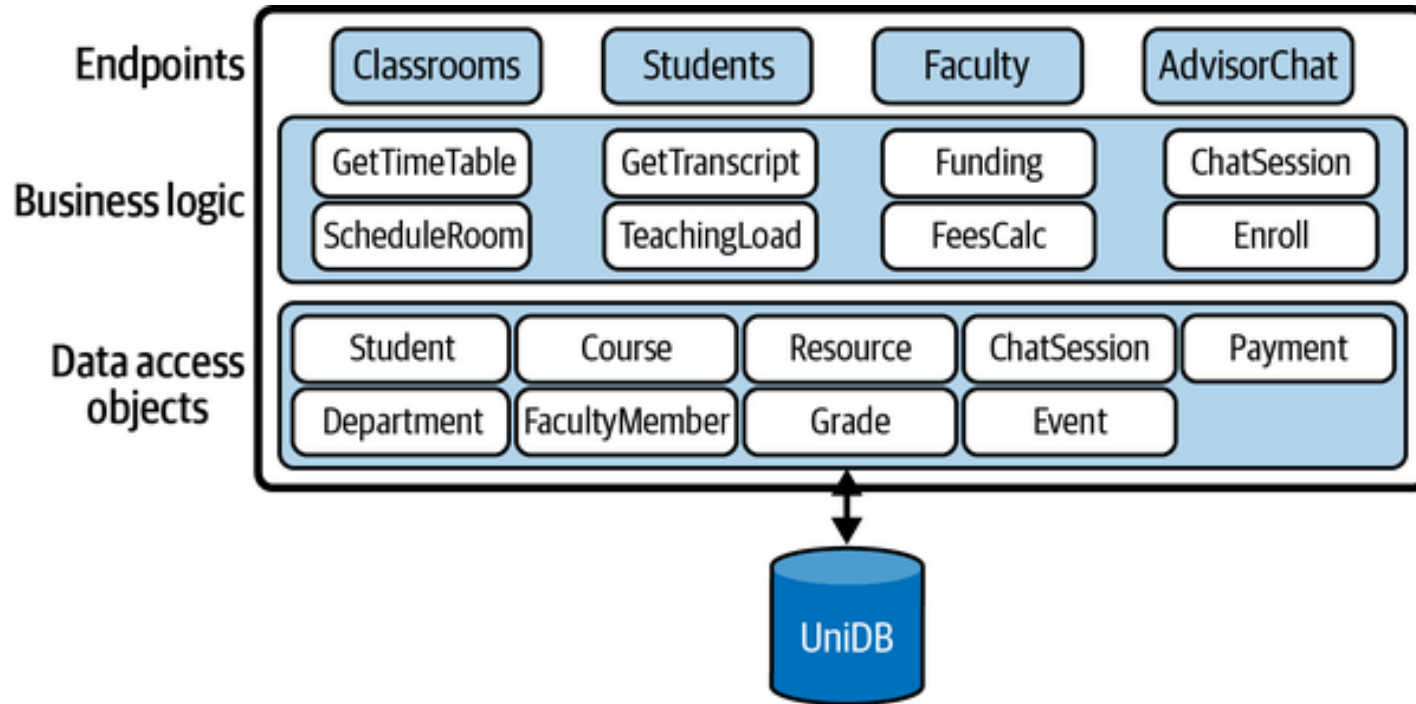


Week 6 – An Architecture for Microservices!!!

- Topics we will cover today

- Monoliths
- Moving to Microservices (this is an oldie but a goodie! Referenced a lot in these slides!)
<https://www.youtube.com/watch?v=PFQnNFe27kU>
 - Note that the video was in the era that was also the DAWN of Docker! (2015)
 - Very exciting times!!!
- Characteristics of a microservices based system
- Resilience!
- Circuit Breaker Pattern
- Bulkhead Pattern

Monoliths! ONE *BIG* PROGRAM!!!



This is from Sam's 2nd edition book on Microservices!

Monoliths

- APIs, business logic, data access, etc. all in a **single** deployment artifact
- Scale up is conceptually simple – buy a bigger box
- Scale out
 - Multiple copies of the monolith *can be* provisioned with a load balancer
- Single (giant) package to test
- Single deployment to manage and monitor

Typical Problems with Monolithic Projects

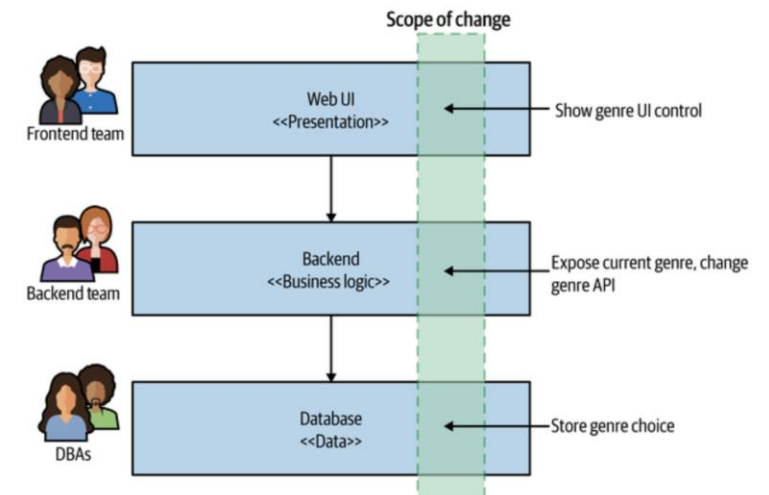
- No internal APIs, integration is in the database
- Difficult to change
 - Time-pressure of projects encourages short-cut connections
 - **Technical debt** grows until change is prohibitively expensive
 - Technology becomes outdated
 - Depressingly low quality => Difficult to retain talent!

Human Organization and Software Development

The structure of the code reflects the structure of the organization

- Three teams will produce three components
- One giant project produces one giant code base with unenforced internal APIs
- Example

“MusicCorp” introduces genre to the app... it requires changes it all 3 layers!



Testing Monoliths, *typically poor techniques!!!!*

- **Manual end-to-end testing only**
 - Okay for small systems that don't live long and don't change
 - Expensive to repeat, therefore change is discouraged
- Changing a library forces re-test of everything, therefore copy & paste dominates
 - Copy & paste means the same bug appears in multiple places ☹️
- Testing is the last phase of the project and is always cut when the schedule slips

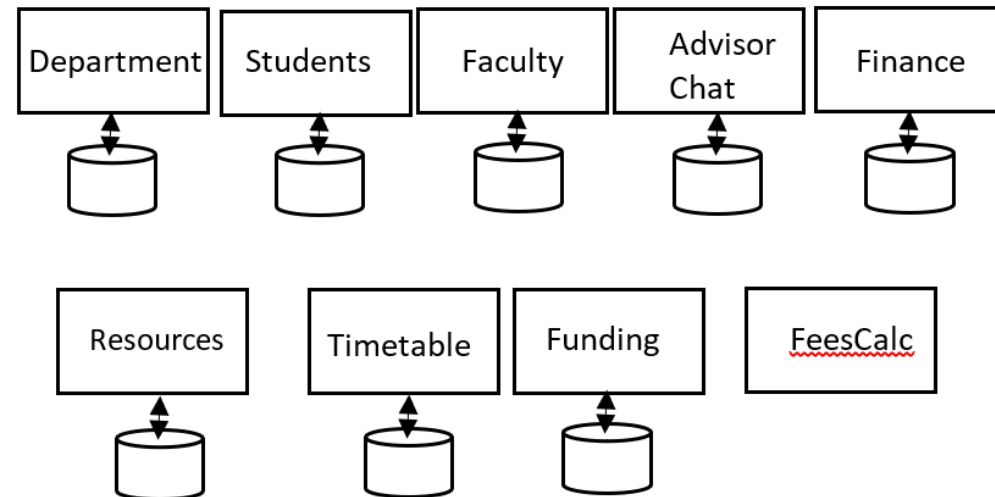
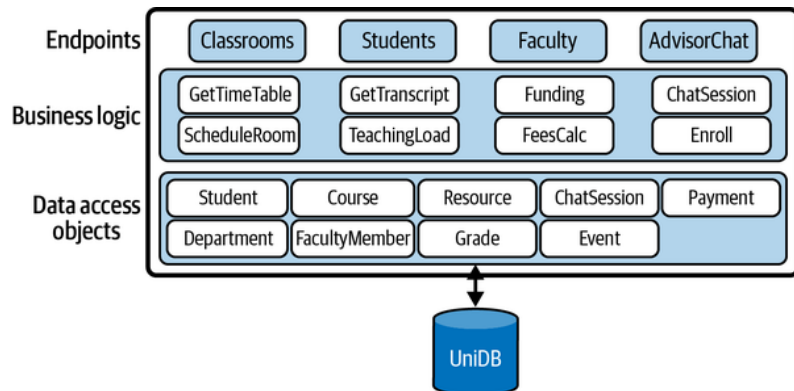
Enter Microservices!!!!

A distributed system with multiple entities talking to one another, while performing their own operations!



Moving to Microservices

- Monoliths are difficult to change: new features, new technology, increasing request volumes
 - **Code base complexity**
 - **Scale Out** makes better use of resources by only scaling services that need it
- *Enter Microservices*
- Breaking up the **Monolith!!!!**



Microservices Advantages

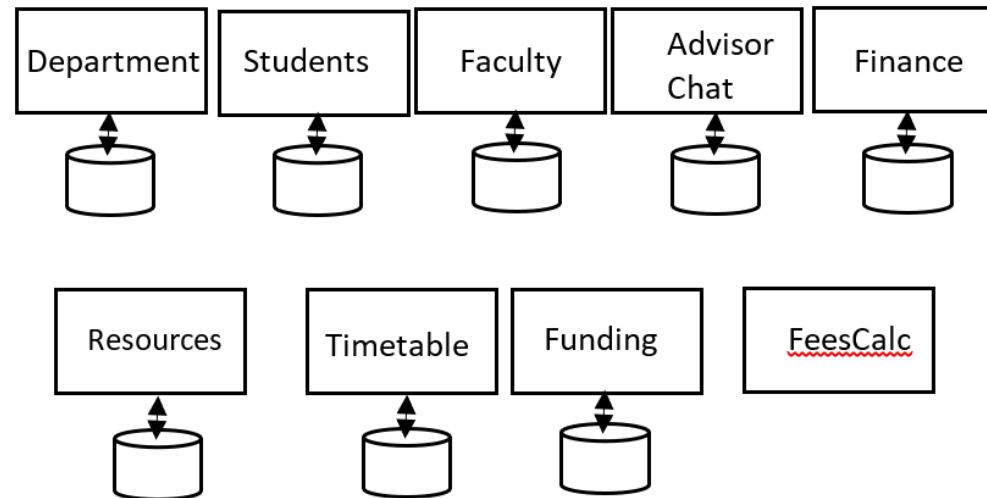
- **Technology Agnostic!!!**
- **Code base:** an individual service should not be more complex than a small team size can build, evolve and manage
 - Project management is simpler!
- **Scale Out:** Individual microservices can be scaled out to meet request volume and latency requirements
 - Only scale the components that need it!
- **Testing:** Easier to test just one “module”
- **Goal:** Build a distributed system with multiple entities talking to one another, while performing their own operations

BUT HOW DO WE BREAK THINGS UP?!?!

- This really is about MODULARITY!
- We had great debates about this back in 70s!
- This was a well known paper on the topic
 - Parnas “On the criteria to be used in decomposing systems into modules”
 - <https://dl.acm.org/doi/10.1145/361598.361623>
 - 1972!!!!

Domain Driven Design – It's about scope!

- Domain-Driven Design (DDD) provides a suitable method for identifying microservices
 - Bounded *Contexts* in DDD (a principle for decomposition!)
 - Major **nouns** in the domain
 - Each owned by a team!!!

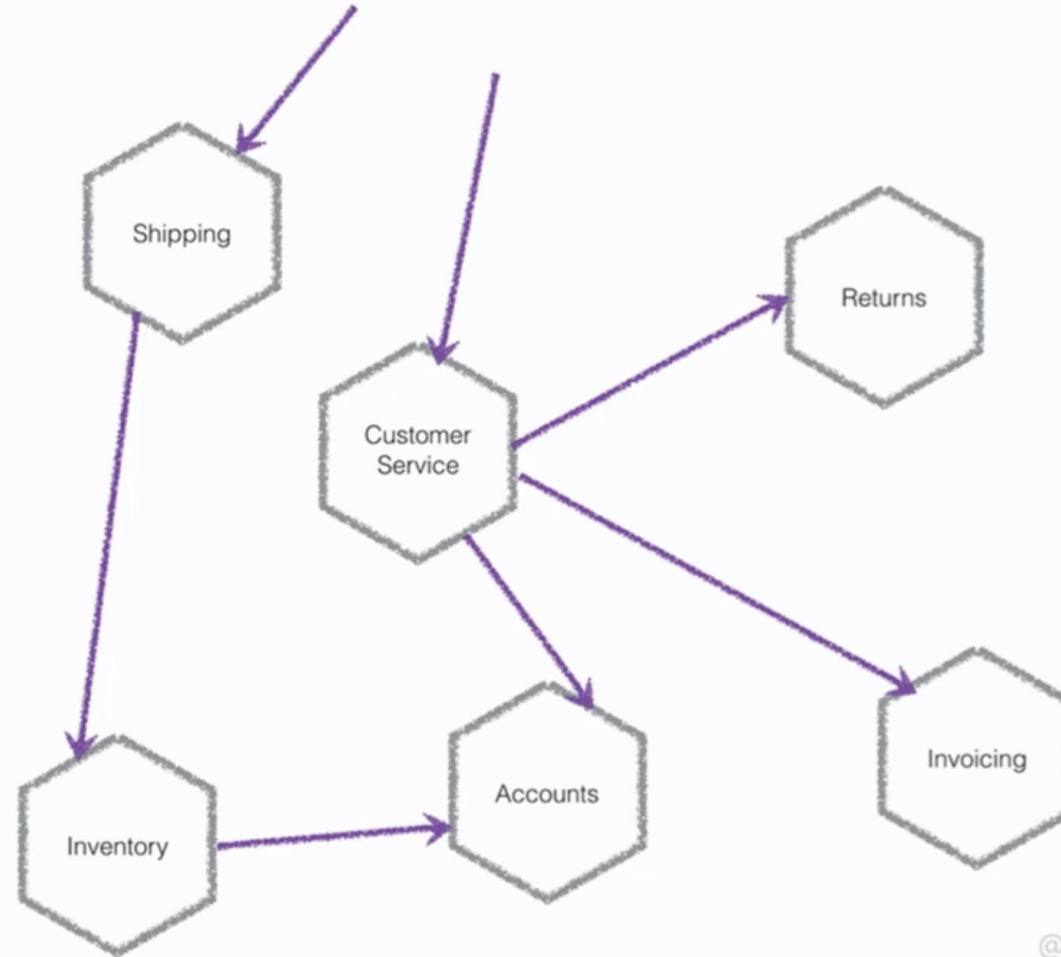


Deploying Microservices Must Be Quick and Easy

- Continuous Deployment & DevOps
- *Containers* - Everything needed to run the service is packaged!
 - the container object: code, run time, system tools, libraries and dependencies
- BUT ALSO *Serverless* – FAAS (functions as a service)
 - Serverless has 3 advantages
 - **Deployment is simple** – just upload the new executable package for your microservice to the endpoint that you have configured for your function
 - **Pay by usage** – if your service has periods of low volume requests, your costs are low, even zero
 - **Ease of Scaling** – the platform you choose handles scaling of your function. You control precisely how this works through configuration parameters, but the serverless option takes the heavy lifting out of scalability!
 - **Examples: AWS Lambda, Azure Functions**

Want them to be well defined and separate!

This depends on the domain!

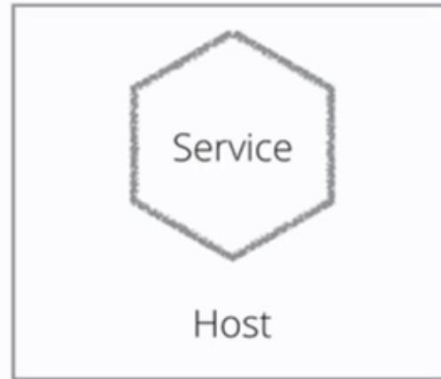


@samnewman

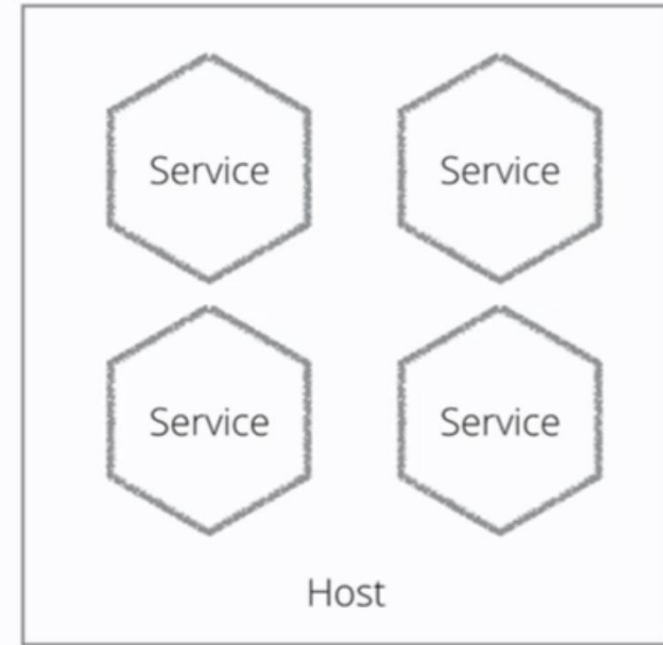
How do services map to hosts?!

Where a host is an **isolated system**!

A physical machine,
a virtual machine
or a container!



VS



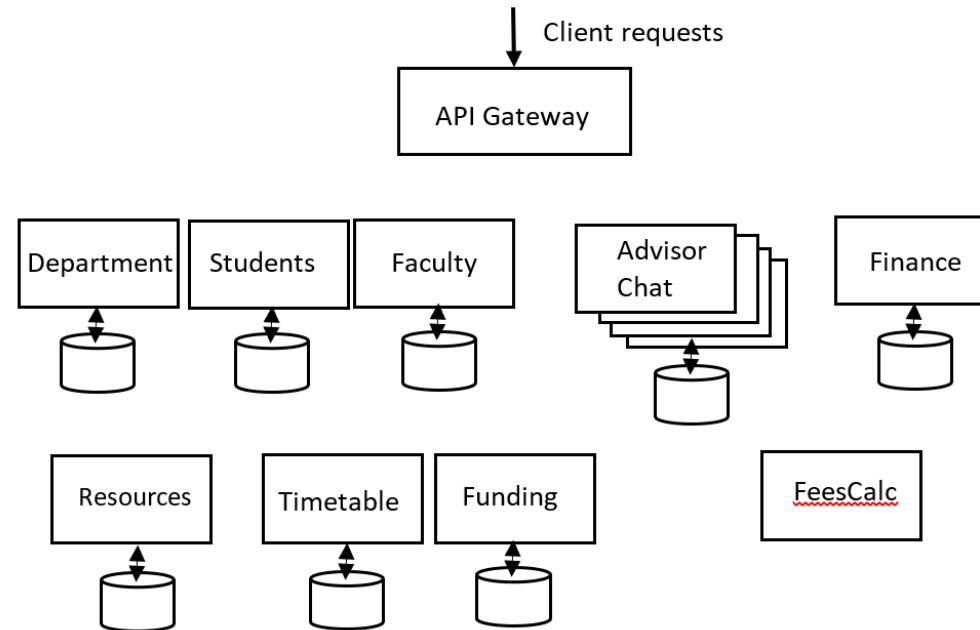
@samnewman

Wait a minute... there are tradeoffs here!

- Multiple endpoints that clients need to invoke...
 - Host, IP address, port for each microservice?!
- What if your refactor?!
 - combining two in or order to eliminate network calls? move an API implementation from one microservice to another? change the endpoint – IP address and port – of an API?
- Gang of Four Design patterns *façade pattern* in object-oriented systems!
 - API Gateway pattern!
 - acts as a single entry point for all client requests
 - insulates clients from the underlying architecture

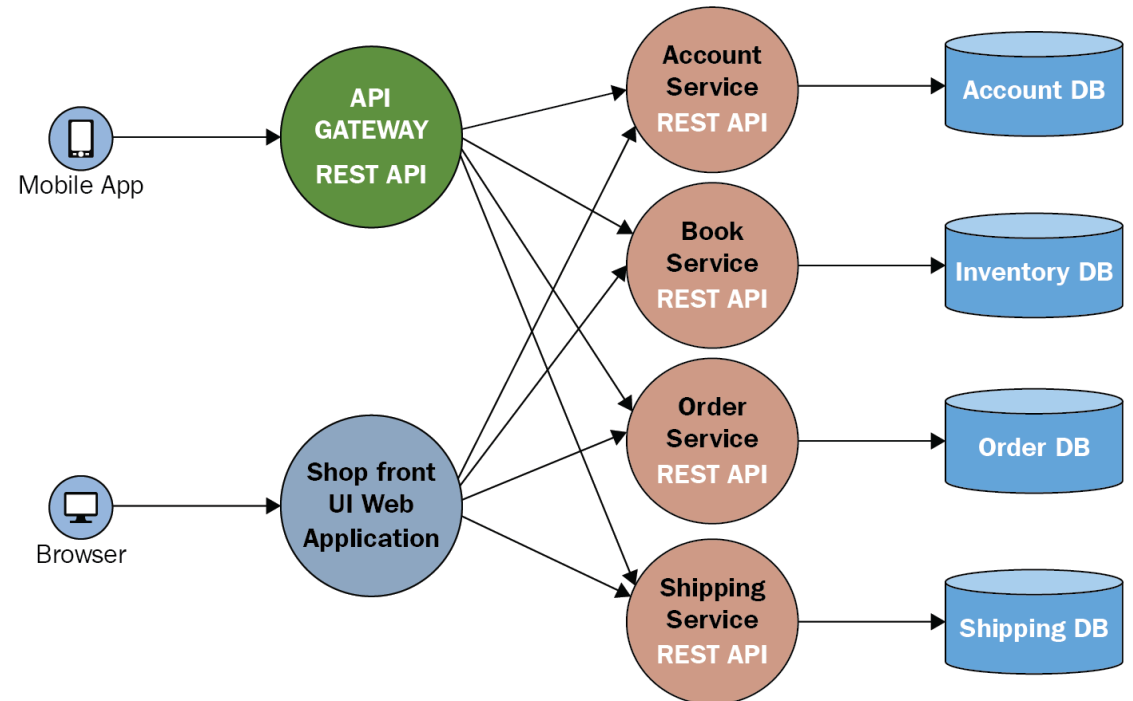
API Gateway – the Façade Pattern

- An API Gateway is a single entry point for all client requests
- *Insulates* clients from the underlying architecture of the microservices that implement the application functionality.
 - Example: <https://aws.amazon.com/api-gateway/>



API Gateways CAN do a lot of things...

- Proxy incoming client API requests with low millisecond latencies to backend
- Provide authentication and authorization
- Define rules for throttling each API
- Support a cache for API results
- Integrate with monitoring tools
- PLUS can be a new bottleneck!!!!

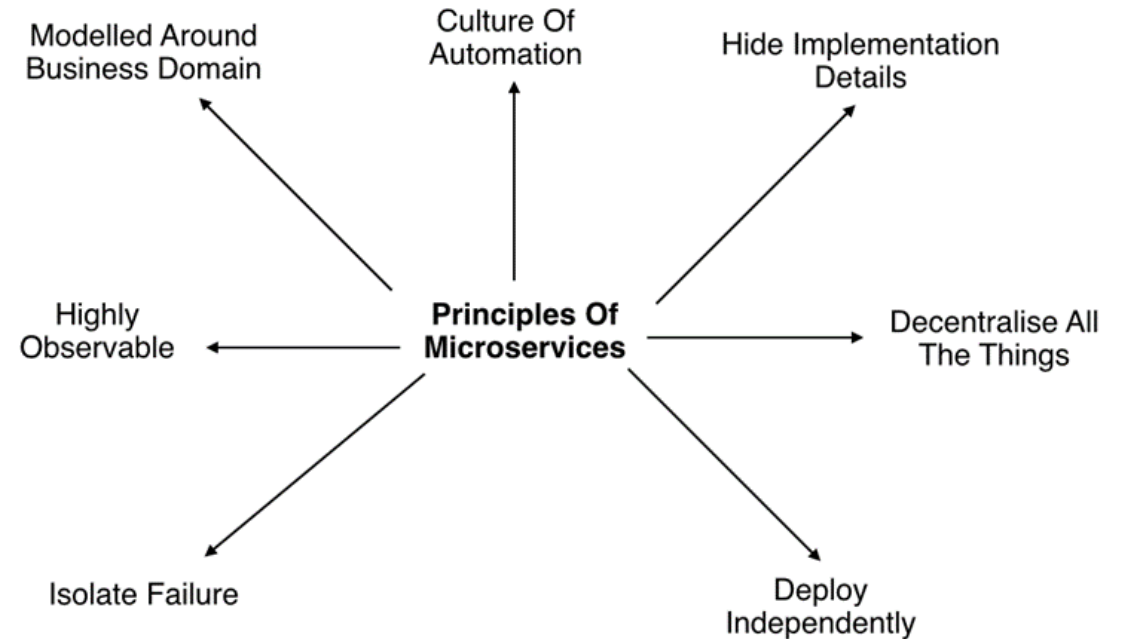


Why the difference between mobile and browser?!

- **Network Constraints:** API Gateways can aggregate multiple backend service calls into a single request
- **Security:** API Gateways provide a centralized point for authentication, authorization
- **Client Simplification:** By offloading concerns like service discovery, load balancing, and error handling
- **Versioning and Evolution:** API Gateways facilitate managing API versions and evolving backend services

Microservice Principles (see *Sam's video!*)

- Modelled around a business domain
- Highly Observable
- Hide Implementation Details
- Decentralize all the things
- Isolate Failure
- Deploy Immediately
- Culture of Automation



Microservices Based Systems are EXCITING!

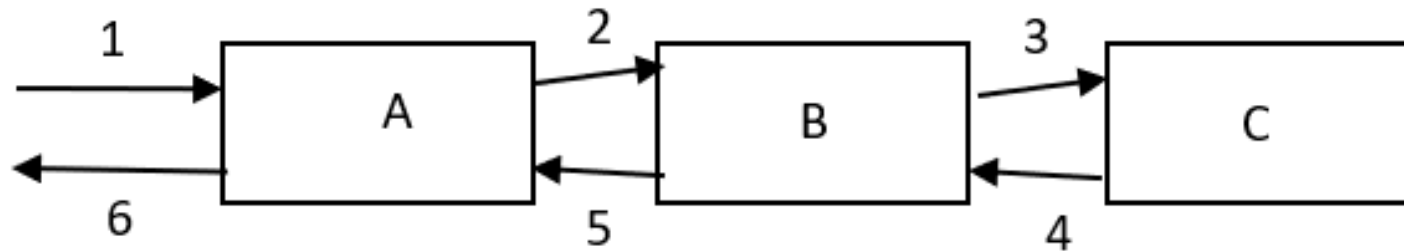
- Applications comprise multiple communicating microservices
- How do we decompose them? (recommended: Domain Driven Design!)
 - Rules?
 - Methods?
- Microservices are (like Parnas said in the 70s?!)
 - Loosely coupled
 - Cohesive
- Microservices typically
 - Single responsibility
 - Small team (5-10)
 - Independently deployable
- Communications ideally will be...
 - Reliable
 - Low latency

Resilience in Microservices

- What should a system do if just one microservice is down?
 - Reject request?
 - Degraded functionality?
 - Example
 - Users logs into music service
 - Personalization microservice that hold user preferences is unavailable?
 - What do we do?
- A **web** of microservices is only as reliable as its weakest component!!!
- PATTERNS to the rescue... AGAIN!

What if one service is slow?

- A downstream service C suddenly slows down
- New requests keep arriving at A
- The upstream service B is waiting on C, then A starts waiting on B
- It keeps waiting for the slow services

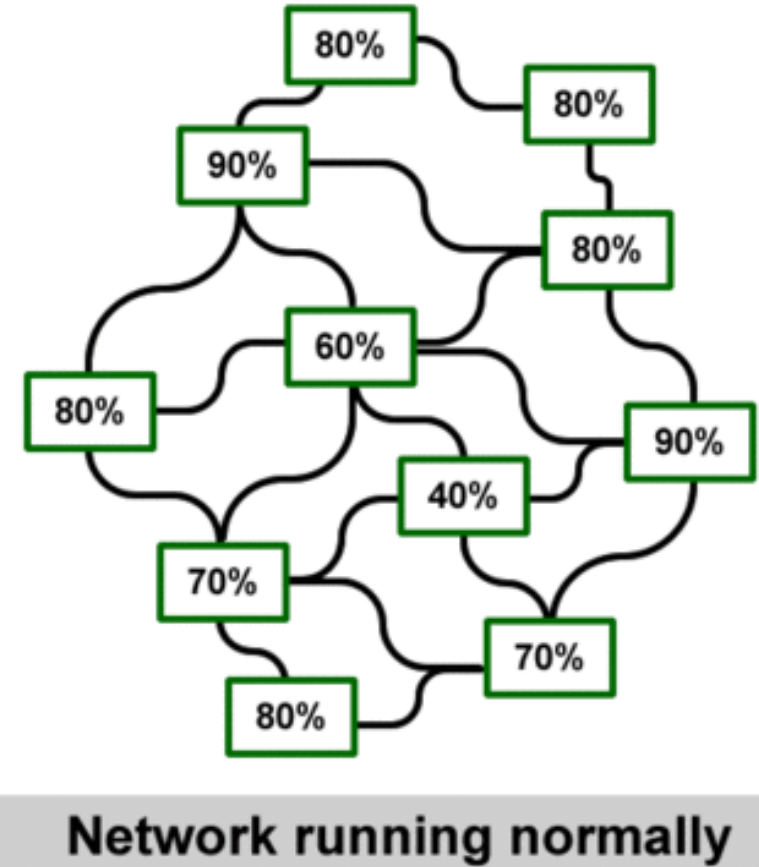


Microservices are “calling” each other...

- If “C” (**downstream** from B and A) is slowing down... responses are slower, BACK PRESSURE on B and A!
- If this is sustained over a period of time...
 - Database has frequently updated keys, and this means updates are one at a time
 - Or network connection to C’s database is unstable?!
- B’s Threads (yes!) are getting blocked
 - Might have a fixed size thread pools!
 - Requests start queuing up!
- Then A’s threads start going through the same thing?!?!

Cascading Failures

- A slow service causes requests to build up upstream
- If we keep sending requests to the slow service
 - Will it get faster?
 - No, it will probably get slower, think of a hashtable
- Requests on upstream service consume threads
 - Block as downstream service slow
 - Runs out of threads
- The whole graph soon becomes unresponsive!

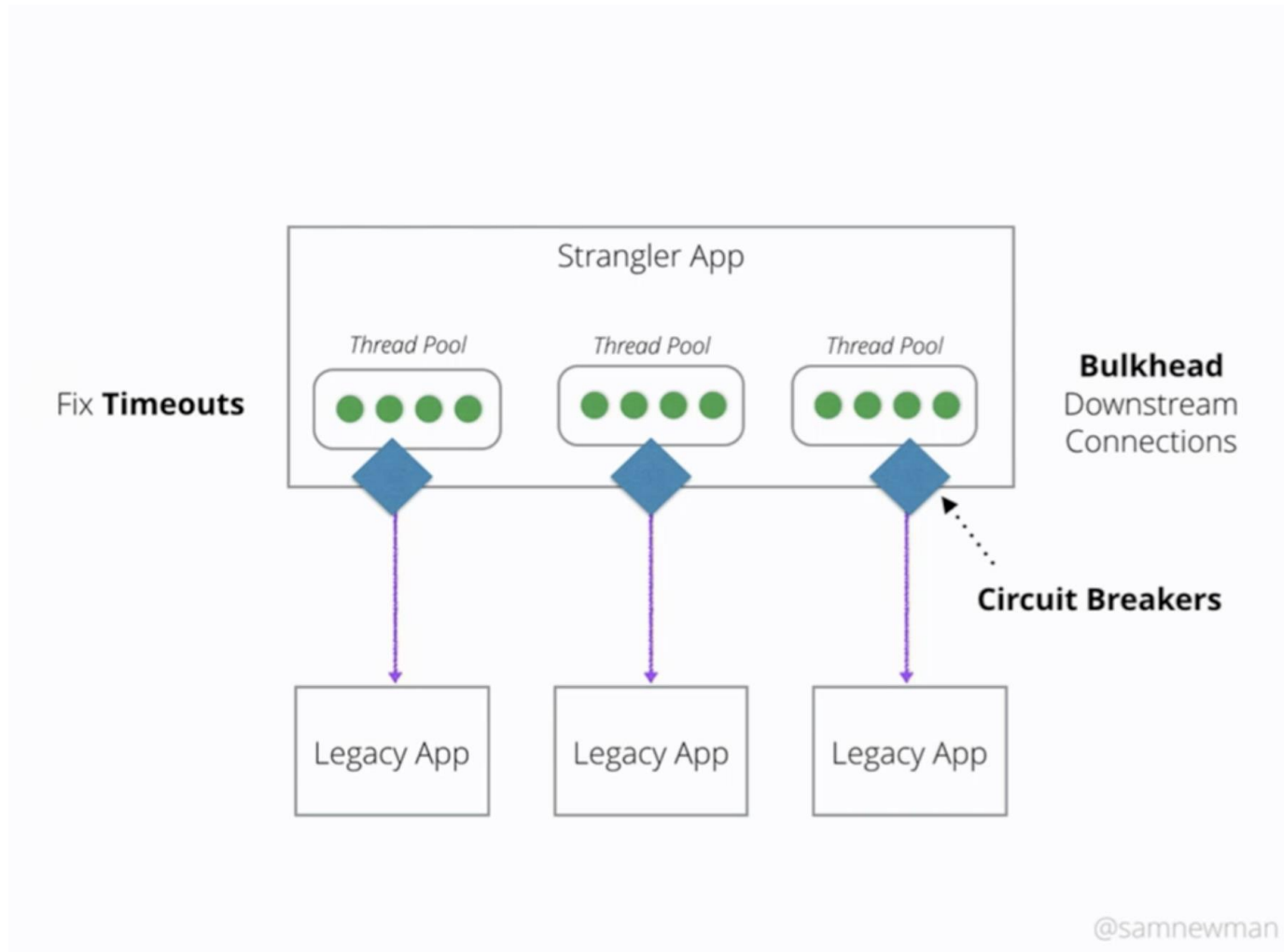


Cascading Failures

- Under heavy load, cascading failures can happen very quickly
- We need to be tolerant to slow services
 - Q: Why are slow services more evil than failed ones?
 - A: Retries don't help
- Detect slow services
- Limit their scope for causing cascading failures

```
int retries = RETRY_COUNT;
while (retries > 0) {
    try {
        callDependentService();
        return true;
    } catch (RemoteCallException ex) {
        logError(e);
        retries = retries - 1;
    }
    return false;
}
```

From Sam's talk, patterns!

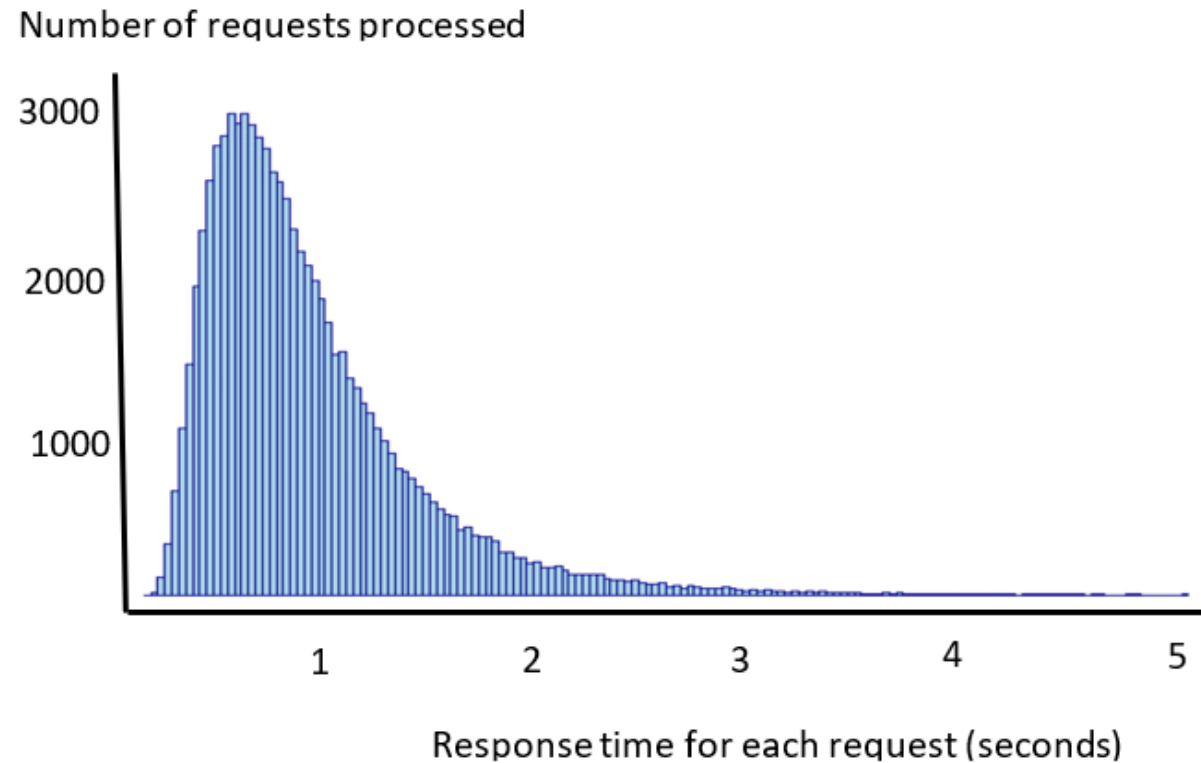


Timeouts – Fail Fast!

- If we wait 'forever' on a slow downstream service
 - We consume resources (thread/connection)
 - Can cause our service to become unresponsive due to blocked threads
- Timeouts on all out of process calls
 - No blocking calls
 - Choose sensible defaults
- If timeout occurs
 - **Log it**
 - Tune timeout settings based on system behaviors
 - Design upstream 'default' functions to deal with failed request

Characteristics of a microservices Based System

- Long Tail Responses



Percentiles

Typical pattern:

- P50 – 200 milliseconds
- P95 – 1200 milliseconds
- P99 – 3000 milliseconds

Solution - fail fast

- When a request takes longer than e.g. p95 (?), client returns an error to its caller
- Enable throttling on a server. If load exceeds threshold, immediately fail the request with an HTTP 503 error.

Circuit Breaker Pattern

- Analogous to electrical system, If a power surge, the breaker blows to protect appliances
- In software if there's a traffic surge or a slow/dead downstream service
 - The breaker blows and stops sending requests
 - For HTTP requests, indicators are 5XX errors & Timeouts
 - Circuit breaker monitors % of failed calls
 - When threshold reached, it blows
- New requests are:
 - Queued if appropriate
 - Fail fast
- Circuit breaker waits a period and starts allowing requests through
 - How long?
 - If still fail?
 - If succeed?

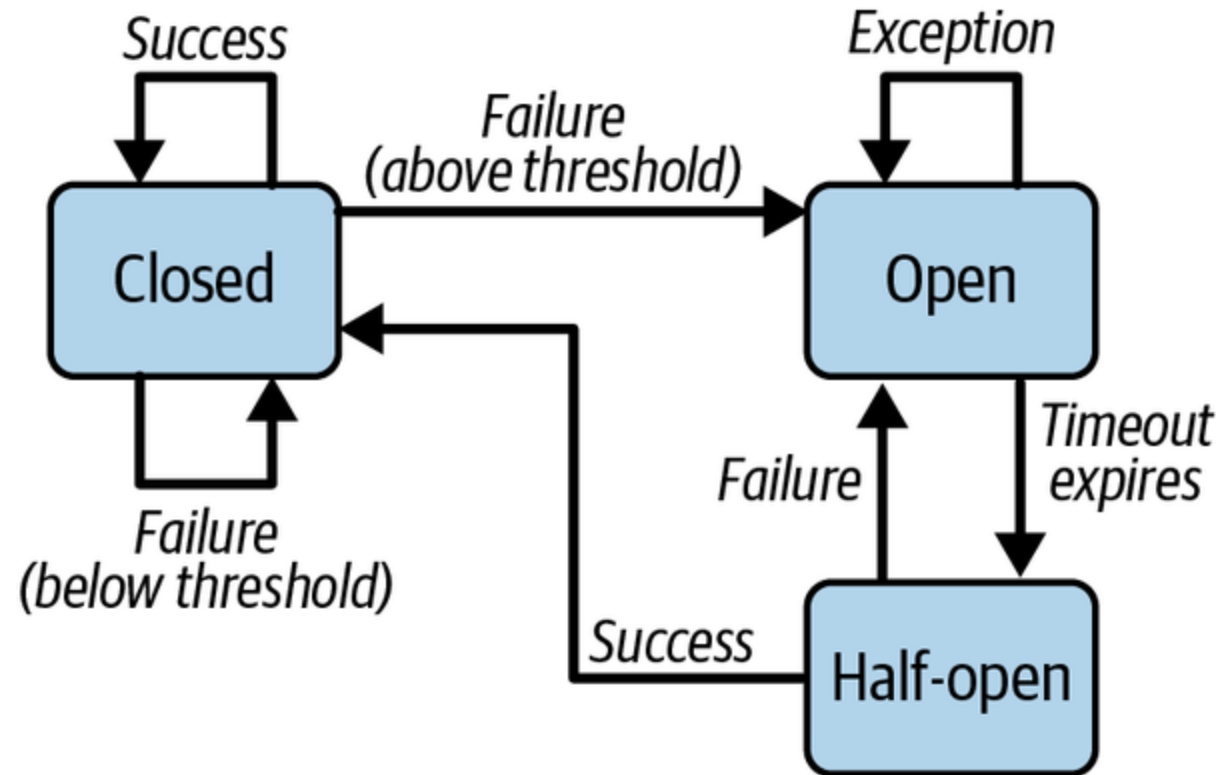


Like a bridge: Open – no traffic. Closed - traffic



Circuit Breaker Pattern

Circuit breaker starts in Closed state, ie the switch is closed and traffic can flow.



Be careful of the meaning, think of a lifting bridge

- Open – the circuit is broken and it is Closed to traffic
- Close – the circuit is complete and it is Open to traffic

Apache EventCountCircuitBreaker

Max load is 1000 per minute.

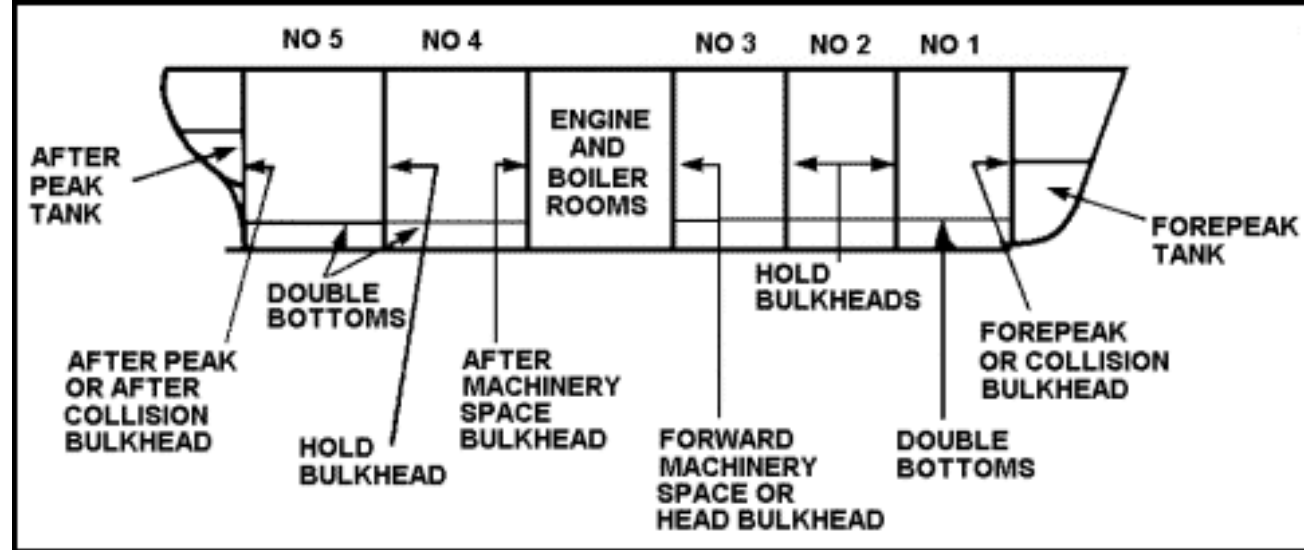
If exceeded, new requests blocked until rate drops below 800.

Use this to limit the incoming calls so that this service is not overloaded

```
EventCountCircuitBreaker breaker =  
new EventCountCircuitBreaker  
    1000, 1, TimeUnit.MINUTE, 800);  
...  
public void handleRequest(Request request) {  
    if (breaker.incrementAndCheckState()) {  
        // actually handle this request, CLOSED  
    } else {  
        // do something else, OPEN  
        // e.g. send an error code  
    }  
}
```

Bulkhead Pattern

- Isolates requests into pools
- If one fails/runs out of resources, others unaffected
- Analogous to ship bulkheads... ?!?! 😊
- If a leak happens, it is limited!



Bulkhead Pattern

Server has **multiple services**

- If one is latent, it will consume all threads
- All services affected by single delayed service

Client may issue requests to many services simultaneously

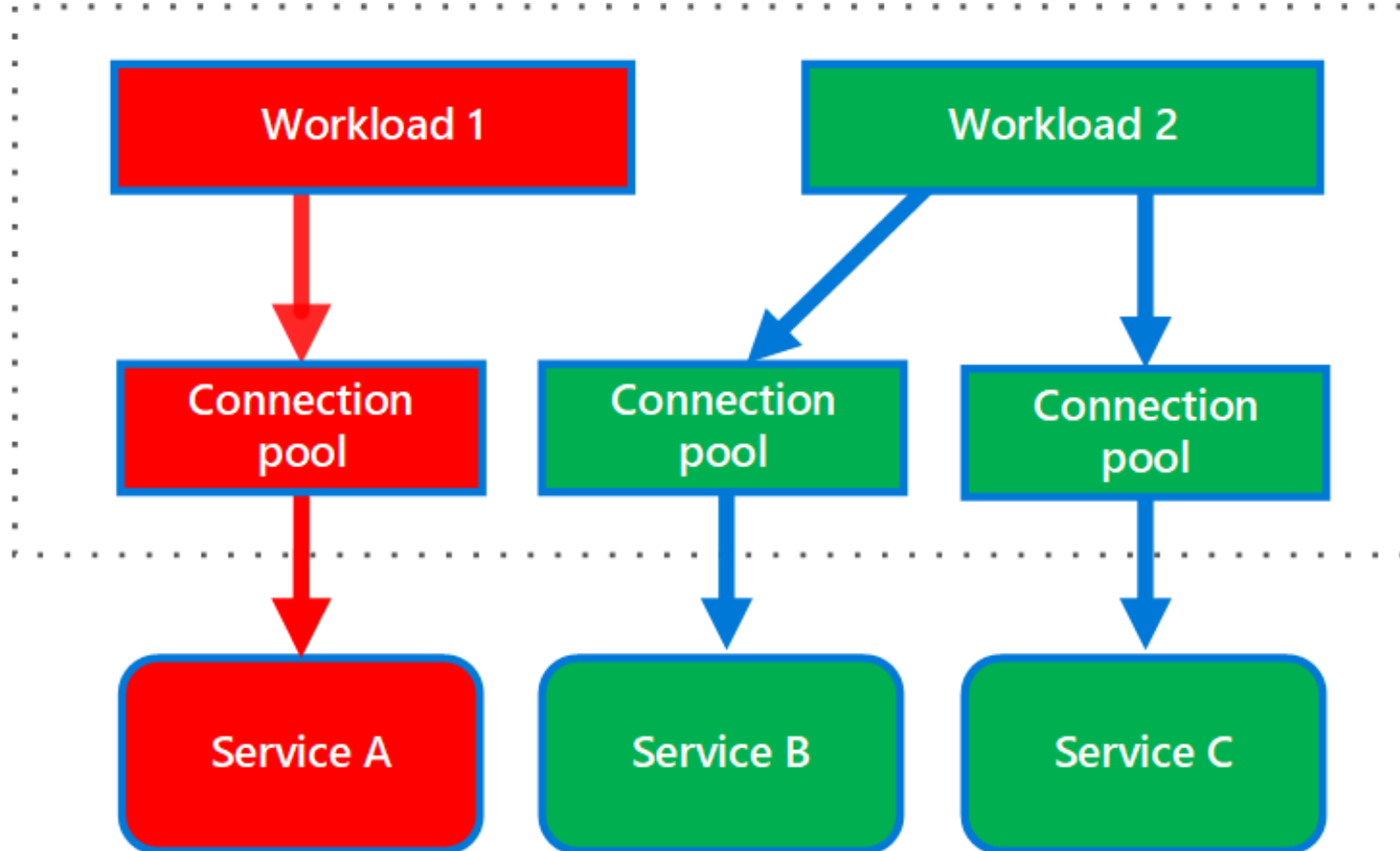
- Latent dependencies block client threads
- Potential for client thread pool exhaustion for all service requests

Scenario!

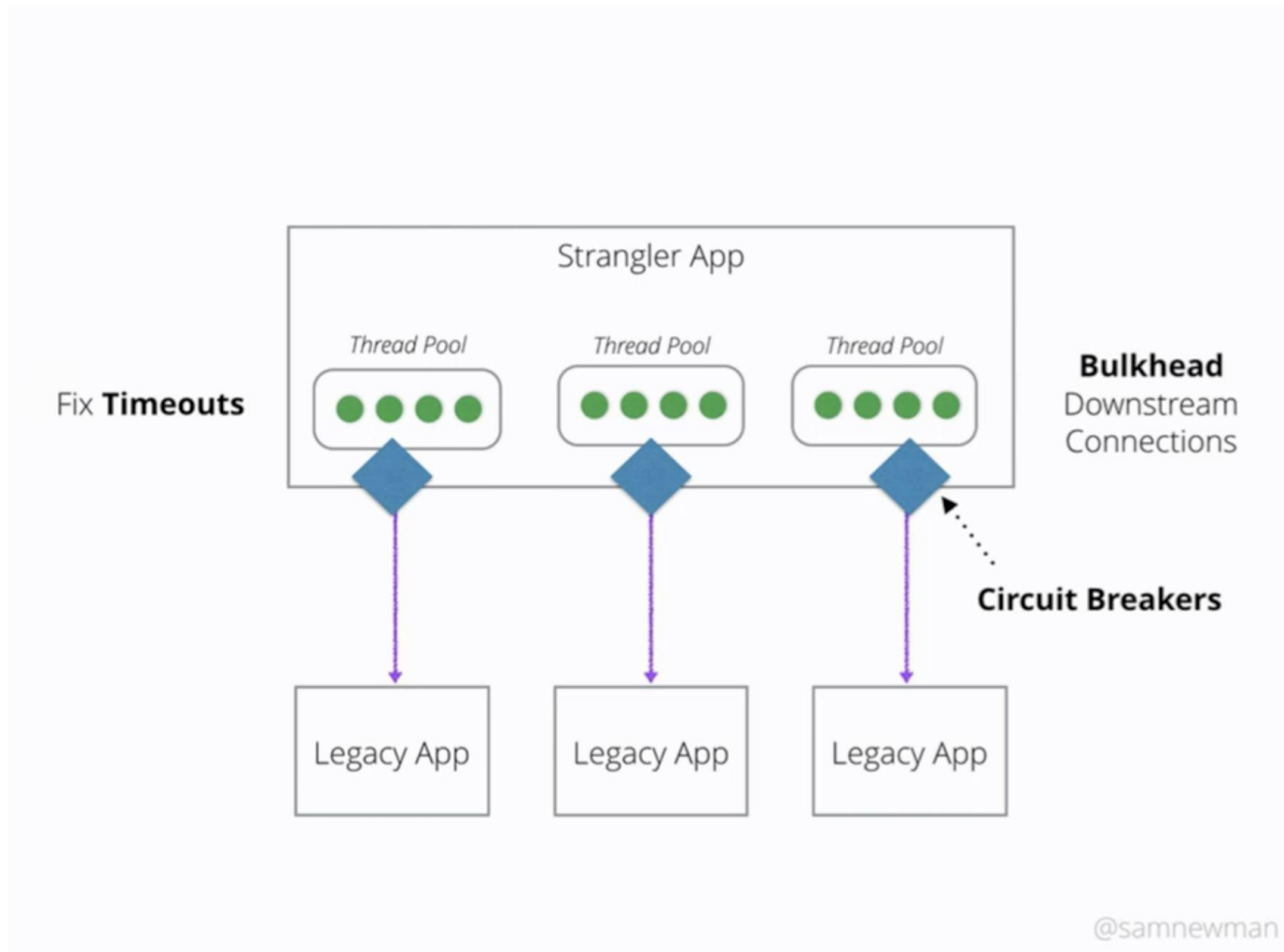
- A microservice with two endpoints
 - A read (current orders?)
 - A write (new orders!)
- Requests for these two endpoints share a common thread pool
 - Suddenly, there is a new order SURGE!
 - All threads run to help!
 - Reads are blocked!
 - If fail fast is used, clients see errors!?!

Bulkhead Pattern

- Reserve a number of threads to handle specific requests!



From Sam's talk, patterns!



Observability... *THIS IS YOUR FOCUS!!!!*

- Log aggregation
- Statistic aggregation
- Tracing how are services connected to each other?!
- Cindy Sridharan's book [Distributed Systems Observability](#)

Observability isn't purely a technical concept that can be achieved by buying an "observability tool" (no matter what any vendor might say) or adopting the open standard du jour. To the contrary, observability is more a sociotechnical concept. Successfully implementing observability depends just as much, if not more, on having the appropriate cultural scaffolding to support the way software is developed, deployed, debugged, and maintained, as it does on having the right tool at one's disposal.

Microservice Summary

- Microservices are a **dominant** architecture for scalable systems
 - Loosely Coupled, cohesive, distributed
 - Resilience in the face of Cascading Failures!!!
 - Circuit Breaker Patterns
 - Bulkhead (Isolation) Pattern
-
- If you see a *monolith*.... DO NOT PANIC, you can handle this! 😊